

NAT (Network Address Translation)

Os hosts internos usam endereços IP privados (**192.168.100.x**) que não existem na internet — nenhum roteador externo sabe como devolver um pacote para esses endereços. Sem NAT, um pacote de **pc1_br1** chegaria ao destino na internet, mas a resposta seria descartada no caminho de volta por falta de rota. Este passo configura o NAT no **r4_edge** para resolver esse problema.

Configuração

Uma única regra **iptables** no **r4_edge** resolve o NAT para toda a rede interna:

```
docker exec r4_edge iptables -t nat -A POSTROUTING -o <interface_internet> -j MASQUERADE
```

Parâmetro	Significado
-t nat	Opera na tabela NAT do iptables
-A POSTROUTING	Aplicado imediatamente antes do pacote sair da máquina
-o <interface>	Apenas para pacotes saindo pela interface de internet
-j MASQUERADE	Reescreve o IP de origem com o IP atual da interface de saída

O **MASQUERADE** é uma forma dinâmica de SNAT (Source NAT). Ele usa automaticamente o IP da interface de saída (**203.0.113.1**), sem precisar hardcodar o endereço. O **r4_edge** mantém internamente uma tabela de conexões ativas para saber como reverter a tradução quando a resposta chegar.

Nota: a interface de internet é detectada em tempo de execução pelo mesmo mecanismo do passo 1 — a interface pode ser **eth0** ou **eth1** dependendo do que o Docker atribuiu nessa execução.

Como o NAT funciona

O fluxo completo de um pacote de **pc1_br1** até **srv_public_web**:

```
1. pc1_br1 envia:      src=192.168.100.66  dst=203.0.113.10
2. r2_br1 encaminha:  src=192.168.100.66  dst=203.0.113.10  (roteamento normal)
3. r4_edge reescreve: src=203.0.113.1      dst=203.0.113.10  → sai para internet
4. srv_public_web responde: src=203.0.113.10  dst=203.0.113.1
5. r4_edge restaura:  src=203.0.113.10      dst=192.168.100.66 → encaminha para r2_br1
6. r2_br1 entrega:    pacote chega em pc1_br1
```

O host interno nunca percebe a tradução — ela é transparente. Da perspectiva do `srv_public_web`, o cliente é sempre `203.0.113.1`, independentemente de qual host interno originou a conexão.

Por que apenas o `r4_edge` faz NAT?

O NAT precisa estar no ponto de fronteira entre a rede privada e a internet. O `r4_edge` é o único roteador com uma interface em cada lado:

```
eth? → net_servico    (192.168.100.0/26) – rede privada
eth? → net_internet   (203.0.113.0/24)  – internet
```

Os demais roteadores (`r1_hq`, `r2_br1`, `r3_br2`) conectam apenas sub-redes privadas entre si e nunca tocam a internet, portanto não precisam de NAT.

Verificação

A verificação foi implementada no script `scripts/04_nat/test.sh` em três níveis.

O **Nível 1** confirma que a regra MASQUERADE existe na tabela NAT do `r4_edge`:

```
docker exec r4_edge iptables -t nat -L POSTROUTING | grep MASQUERADE
# Output: MASQUERADE all -- anywhere anywhere
```

O **Nível 2** testa a conectividade de todos os hosts internos com `srv_public_web` (203.0.113.10) via `ping`. Um timeout indica que o NAT não está traduzindo corretamente.

O **Nível 3** executa um `traceroute` de `pc1_br1` até `srv_public_web` para confirmar o caminho completo:

```
traceroute to 203.0.113.10, 30 hops max
 1  192.168.100.65  ← r2_br1 (gateway de Polo 1)
 2  192.168.100.4   ← r4_edge (backbone)
 3  203.0.113.10    ← srv_public_web (destino)
```

O salto 2 mostra o `r4_edge` pelo seu IP do backbone (192.168.100.4). A tradução NAT acontece nesse ponto — após o salto 2, o pacote sai com IP de origem `203.0.113.1`, mas o `traceroute` não mostra isso porque o ICMP TTL exceeded é gerado antes da reescrita.

Resultado: todos os 8 testes passaram — `ALL 8 checks PASSED`.